چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۵

ماژول‌بندی و Dynamic Feature — تفکیک api و impl، گراف وابستگی و Play Feature Delivery



اسماعیل مرادی

@esmaeeil-moradi700

چرا ماژول‌بندی در ۲۰۲۶ مهم است؟

یک مونولیت به‌صورت یک واحد کامپایل، تست و review می‌شود. ماژول‌ها قواعد معماری را به **قواعد کامپایلر** تبدیل می‌کنند:

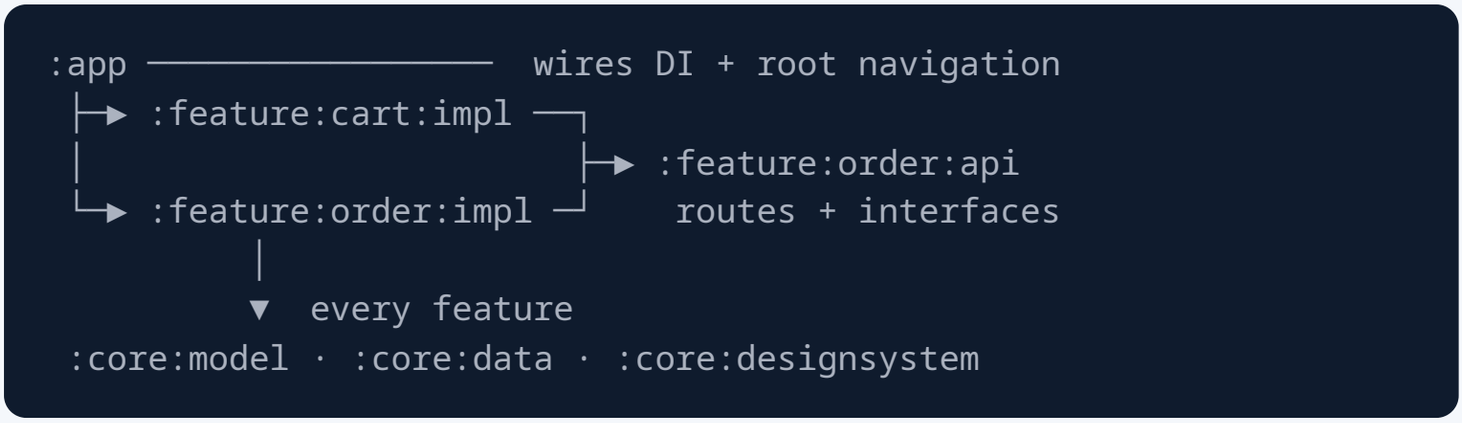
- **سرعت build:** هر تغییر فقط ماژول خودش و وابسته‌هایش را دوباره می‌سازد؛ ماژول‌های مستقل موازی build می‌شوند و از build cache می‌آیند
- **مرزهای اجباری:** `internal` و `implementation` جملهٔ «به آن دست نزن» را به خطای کامپایل تبدیل می‌کنند
- **مالکیت:** هر تیم مالک یک ماژول، API آن و reviewهایش است
- **استفادهٔ دوباره:** همان ماژول‌ها در اپ موبایل، اپ دمو، Wear OS یا KMP
- **تحویل:** Play Feature Delivery می‌تواند بخش‌هایی از اپ را بعداً نصب کند — با هزینه‌ای واقعی

سطح	سؤال‌ها
جونیور	چرا ماژول‌بندی، انواع ماژول، جهت وابستگی
مید	تفکیک api و impl، ناوبری بین فیچرها، DI بین ماژول‌ها، resourceها و ماژول‌های core
سینیور	سرعت build: واقعاً چه چیزی سریع‌تر می‌شود، Play Feature Delivery، هزینه‌ها و جایگزین‌هایش
آرکیتکت	حاکمیت بر گراف، مهاجرت از مونولیت، دانه‌بندی و اپ‌های دمو



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: فیچرها از طریق قرارداد با هم حرف می‌زنند



- **ماژول `app` لاغر است:** `Application`، گراف DI و ناوبری ریشه — بدون منطق فیچر
- **فیچرها از طریق `api` حرف می‌زنند:** routeها و interfaceها، هرگز `impl` یک فیچر دیگر
- **ماژول‌های core مشترک و پایدارند:** هیچ‌وقت به فیچرها وابسته نمی‌شوند

قاعدهٔ پشت هر کدبیس ماژولار

به چیزی وابسته شوید که کم تغییر می‌کند (قراردادها، core)، هرگز به چیزی که هر روز تغییر می‌کند (جزئیات داخلی یک فیچر دیگر). آن‌وقت هر تغییر داخل یک ماژول می‌ماند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

چرا ماژول‌بندی، انواع ماژول، جهت وابستگی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چرا ماژول‌بندی — و هزینه‌اش چیست

چه به دست می‌آورید	چه می‌پردازید
build افزایشی فقط ماژول‌های تحت‌تأثیر را دوباره کامپایل می‌کند	سربار configuration و taskها برای هر ماژول
ماژول‌های مستقل موازی build می‌شوند و از build cache می‌آیند	فایل‌های build بیشتر که باید یکدست بمانند (convention plugin، روز ۲۱)
کپسوله‌سازی واقعی با `internal` و `implementation`	قرارداد بین ماژول‌ها باید طراحی شود
مالکیت روشن، merge conflict کمتر	refactor بین چند ماژول قدم‌های بیشتری دارد
هر فیچر جداگانه build، تست و اجرا می‌شود	ناوبری و DI باید صریحاً سیم‌کشی شوند

تلهٔ مصاحبه

«*ماژول‌بندی همیشه build را سریع‌تر می‌کند.*» — build **افزایشی** و buildهای CI با cache را سریع‌تر می‌کند. build تمیز (clean) با تعداد زیادی ماژول کوچک اندرویدی حتی ممکن است کندتر شود: هر ماژول configuration و taskهای resource، manifest و lint خودش را دارد. قبل و بعد را اندازه بگیرید.

سؤال تکمیلی

«*چرا `internal` دلیلی برای ماژول‌بندی است؟*» — در Kotlin، `internal` یعنی *قابل‌دیدن فقط داخل همین ماژول Gradle*. در مونولیت هر کلاسی از همه‌جا در دسترس است؛ در ماژول‌ها کامپایلر کلاس‌های پیاده‌سازی را پنهان می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. انواع ماژول

نوع	محتوا	وابسته به
`:app`	`Application`، ناوبری ریشه، سیم‌کشی DI	فیچرها، core
فیچر	صفحه‌ها و ViewModelهای یک فیچر کاربرمحور	core و `:api` فیچرهای دیگر
data	repositoryها و data sourceهای یک حوزهٔ domain	core (مدل، شبکه، دیتابیس)
core	design system، مدل‌ها، شبکه، ابزارها	فقط ماژول‌های core دیگر
تست	fakeها، ruleهای تست، fixtureها (`:core:testing`)	ماژول‌هایی که fake می‌کنند
dynamic feature	کدی که Play می‌تواند بعداً تحویل دهد	ماژول پایهٔ `:app` — **وارونه**

- کدی که به هیچ API اندرویدی نیاز ندارد به **ماژول‌های Kotlin/JVM خالص یا KMP** می‌رود: task کمتر، build سریع‌تر، آماده برای اشتراک

تلهٔ مصاحبه

«کد مشترک را در یک ماژول `:core` بگذارید که همه به آن وابسته‌اند.» (توصیهٔ نسخهٔ پارسال) — تبدیل به کشوی خرت‌وپرت می‌شود: هر تغییرش کل گراف بالای خودش را بی‌اعتبار می‌کند. بر اساس هدف تقسیمش کنید: `:core:model`، `:core:network`، `:core:designsystem`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. جهت وابستگی

- وابستگی‌ها **یک‌طرفه‌اند**: از app به feature، از feature به data، و از data به سمت core می‌روند. لایهٔ core هیچ فیچری را نمی‌شناسد.
- یک فیچر می‌تواند به `:api` فیچر دیگر وابسته باشد — هرگز به `:impl` آن.
- فقط `:app` (یا اپ دموی یک فیچر) به ماژول‌های `:impl` وابسته است.

```
// feature/cart/impl/build.gradle.kts
plugins {
    id("app.android.feature") // convention plugin (Day 21)
}

dependencies {
    implementation(project(":feature:order:api"))
    implementation(project(":core:data"))
    implementation(project(":core:designsystem"))
    // implementation(project(":feature:order:impl")) — never
}
```

تلهٔ مصاحبه

«ساختار لایه‌ای app و feature و data و core جلوی وابستگی چرخه‌ای را می‌گیرد.» — خود Gradle روی چرخه بین projectها build را fail می‌کند. آسیب واقعی از یال‌های **مجاز اما بد** می‌آید: فیچری که به `:impl` فیچر دیگر وابسته است، یا همه‌چیز وابسته به یک `:core` غول‌پیکر. گراف را در CI چک کنید (سؤال ۱۱).



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

تفکیک api و impl، ناوبری بین فیچرها، DI بین ماژول‌ها، resourceها و ماژول‌های core



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. تفکیک api و impl

```
:feature:order:api       small · stable · rarely changes
  OrderRoute, interface OrderRepository
        ▲ implements                ▲ uses
:feature:order:impl          :feature:cart:impl
  screens, ViewModels,
  DefaultOrderRepository, DI module
```

ماژول `:api`	ماژول `:impl`
routeها، interfaceهای عمومی و مدل‌هایی که استفاده می‌کنند	صفحه‌ها، ViewModelها، repositoryها، bindingهای DI
وابستگی کم — بدون Room، بدون Compose UI	هر چه لازم دارد
کم تغییر می‌کند، پس مصرف‌کننده‌ها به‌ندرت دوباره کامپایل می‌شوند	هر روز تغییر می‌کند — خودش build می‌شود، `:app` دوباره بسته‌بندی می‌شود، بقیهٔ فیچرها دست‌نخورده



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. در کد چه شکلی است

```
// :feature:order:api
@Serializable data class OrderRoute(val orderId: String)

interface OrderRepository {
    fun observeOpenCount(): Flow<Int>
}

// :feature:order:impl
internal class DefaultOrderRepository @Inject constructor(
    private val dao: OrderDao,
) : OrderRepository {
    override fun observeOpenCount(): Flow<Int> =
        dao.observeOpenCount()
}
```

تلهٔ مصاحبه

«ماژول `:api` یعنی استفاده از configurationای به اسم `api()` در Gradle.» — ربطی ندارند. `:api` یک قرارداد نام‌گذاری ماژول است. انتخاب بین `api()` و `implementation()` تعیین می‌کند که یک وابستگی به compile classpath مصرف‌کننده‌ها نشت کند یا نه (روز ۲۲). داخل هر دو ماژول، `implementation` را ترجیح دهید.

تلهٔ مصاحبه

«هر ماژولی باید یک جفت api و impl داشته باشد.» — فقط ماژول‌هایی که فیچرهای دیگر مصرفشان می‌کنند. تفکیک همه‌چیز، تعداد ماژول‌ها و سربارشان را دو برابر می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. ناوبری بین ماژول‌های فیچر

```
// :feature:cart:impl — knows nothing about orders
fun NavGraphBuilder.cartScreen(onCheckedOut: (String) -> Unit) {
    composable<CartRoute> {
        CartScreen(onCheckedOut = onCheckedOut)
    }
}

// :app — the only place that connects features
NavHost(navController, startDestination = CartRoute) {
    cartScreen(onCheckedOut = { orderId ->
        navController.navigate(OrderRoute(orderId))
    })
    orderScreen(onBack = { navController.popBackStack() })
}
```

- هر فیچر یک **route** و یک **extension** روی `NavGraphBuilder` منتشر می‌کند؛ `:app` فیچرها را با lambda به هم وصل می‌کند — پس فیچرها اصلاً به هم وابسته نیستند
- اگر فیچری باید مستقیماً فیچر دیگری را باز کند، به route داخل `:api` آن وابسته می‌شود — هرگز به صفحه‌هایش



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. مقیاس دادن

- تیم‌های زیاد: هر `:impl` **گراف خودش را** از طریق DI multibinding اضافه می‌کند و `:app` گراف ریشه را سرهم می‌کند (روز ۲، سؤال ۱۱)
- Navigation 3 هم همین تفکیک را نگه می‌دارد: فیچرها entryها را فراهم می‌کنند و back stack در اختیار اپ است
- آرگومان‌های route در saved state مربوط به back-stack entry می‌مانند، پس در دو طرف مرز ماژول از process death عبور می‌کنند

تلهٔ مصاحبه

«*URIهای deep link را قرارداد بین ماژول‌ها کنید.*» — رشته‌ها در زمان کامپایل چک نمی‌شوند: تغییر نام یک مسیر یا آرگومان فقط در زمان اجرا می‌شکند. داخل اپ از routeهای type-safe در `:api` استفاده کنید؛ URI را برای deep linkهای واقعی بیرونی نگه دارید.

سؤال تکمیلی

«*flowهایی که چند فیچر را درگیر می‌کنند، مثل checkout، کجا زندگی می‌کنند؟*» — در یک nested graph که مالکش یک فیچر است و فقط با یک route در `:api` آن وارد می‌شوید. فیچرهای دیگر فقط همان route ورودی را می‌شناسند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. DI بین ماژول‌ها

```
// :feature:order:impl — the binding lives with the impl
@Module
@InstallIn(SingletonComponent::class)
abstract class OrderDataModule {
    @Binds
    internal abstract fun bindOrderRepository(
        impl: DefaultOrderRepository,
    ): OrderRepository
}

// :feature:cart:impl — sees only :feature:order:api
@HiltViewModel
class CartViewModel @Inject constructor(
    private val orders: OrderRepository,
) : ViewModel()
```

- Hilt کامپوننت‌ها را در `:app` تولید می‌کند، پس هر ماژول Hilt باید در وابستگی‌های (گذرای) `:app` باشد؛ aggregating task در Hilt Gradle plugin باعث می‌شود یال‌های معمولی `implementation` کافی باشند
- در Koin: هر ماژول یک `module { … }` منتشر می‌کند و `:app` فهرستشان را به `startKoin { modules(…) }` می‌دهد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. dynamic feature و وابستگی پنهان

تلهٔ مصاحبه

«*Hilt در ماژول dynamic feature هم همان‌طور کار می‌کند.*» — نمی‌کند: فیچر به اپ وابسته است، نه برعکس، پس Hilt نمی‌تواند کامپوننت‌هایش را تولید کند. در فیچر از Dagger خالص استفاده کنید و وابستگی‌های اپ را از طریق یک `@EntryPoint` بگیرید.

```
// :app
@EntryPoint
@InstallIn(SingletonComponent::class)
interface VideoCallDependencies {
    fun sessionRepository(): SessionRepository
}

// :videocall (dynamic feature)
val deps = EntryPointAccessors.fromApplication(
    applicationContext, VideoCallDependencies::class.java,
)
DaggerVideoCallComponent.builder()
    .videoCallDependencies(deps).build().inject(this)
```

تلهٔ مصاحبه

«*یک event bus سراسری ماژول‌ها را کاملاً از هم جدا می‌کند.*» (توصیهٔ نسخهٔ پارسال) — در واقع وابستگی را پنهان می‌کند: دو طرف همچنان در نوع eventها و زمان‌بندی شریک‌اند، اما هیچ‌جا دیده نمی‌شود چه کسی به چه کسی وابسته است. یک interface در `:api` یا state منتشرشده به شکل `Flow` را ترجیح دهید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. resourceها و ماژول‌های UI مشترک

- هر ماژول `namespace` خودش را دارد و — از AGP 8.0 به‌طور پیش‌فرض — یک **کلاس R غیرگذرا (non-transitive)**: کلاس `R` فقط resourceهای همان ماژول را دارد
- ماژول `:core:designsystem` شامل theme، کامپوننت‌ها و آیکون‌هاست و هیچ مدل domain را نمی‌شناسد؛ ماژول `:core:ui` شامل composableهای مشترکی است که مدل‌های domain را می‌شناسند
- رشته‌های هر فیچر داخل همان فیچر می‌مانند: به‌روزرسانی ترجمه فقط یک ماژول را دوباره می‌سازد
- اسم یکسان resource در دو ماژول: اپ بی‌صدا بر کتابخانه‌ها غلبه می‌کند. در ماژول کتابخانه `resourcePrefix` تعیین کنید تا Lint اسم‌های بدون پیشوند را گزارش کند

```
// core/designsystem/build.gradle.kts
android {
    namespace = "com.app.core.designsystem"
    resourcePrefix = "ds_"
}

// any feature
import com.app.core.designsystem.R as DsR
Text(stringResource(DsR.string.ds_retry))
```

تلهٔ مصاحبه

«*مقدارها را با propertyهای Gradle و* `resValue` *به اشتراک بگذارید.*» (توصیهٔ نسخهٔ پارسال) — این config زمان build است، نه راهبرد resource: نه ترجمه دارد، نه ابزار. و «API key» که این‌طور تزریق شود داخل APK قابل‌خواندن است.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

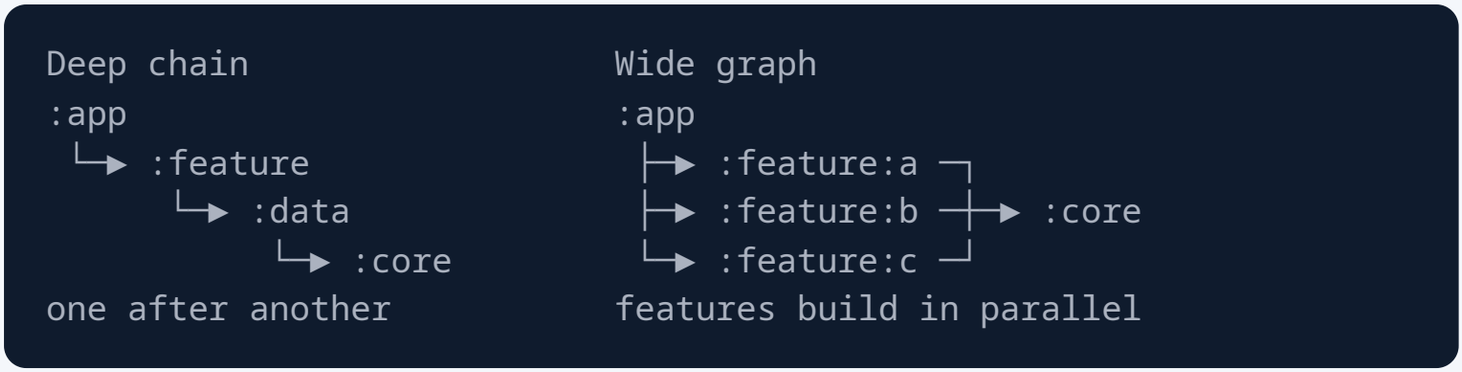
سرعت build: واقعاً چه چیزی سریع‌تر می‌شود، Play Feature Delivery، هزینه‌ها و جایگزین‌هایش



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۸. سرعت build: واقعاً چه چیزی سریع‌تر می‌شود



سازوکار	پیش‌نیاز
compile avoidance: وقتی ABI یک ماژول تغییر نکند، وابسته‌هایش دوباره کامپایل نمی‌شوند	تغییر پشت یک API عمومی پایدار؛ یال‌های `implementation`
اجرای موازی ماژول‌های مستقل	گرافی پهن و کم‌عمق؛ `org.gradle.parallel=true`
build cache: ماژول‌های تغییرنکرده از cache محلی یا remote می‌آیند	taskهای cacheپذیر؛ یک remote cache که CI پرش می‌کند
configuration cache: buildهای تکراری مرحلهٔ configuration را رد می‌کنند	pluginهای سازگار (روز ۲۱)

اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. زمان کجا می‌رود

- **طولانی‌ترین مسیر** گراف کف زمان build است: یک زنجیرهٔ عمیق، هر چقدر هم هسته داشته باشید، پشت‌سرهم build می‌شود
- تغییر امضای عمومی در یک ماژول سطح پایین در همهٔ ماژول‌های بالاتر موج می‌زند — APIهای core را **کوچک و پایدار** نگه دارید
- ماژول‌های کتابخانهٔ اندرویدی خیلی بیشتر از ماژول‌های Kotlin/JVM خالص task دارند
- در همهٔ ماژول‌ها KSP به‌جای kapt (روز ۲۱)
- اندازه بگیرید: Build Analyzer، Build Scan، و `gradle-profiler` برای بنچمارک تکرارپذیر

تلهٔ مصاحبه

«*یک فایل را تغییر بدهید، فقط ماژول خودش دوباره کامپایل می‌شود.*» (ادعای نسخهٔ پارسال) — فقط تا وقتی ABI آن ثابت بماند. یک امضای عمومی در `:core:model` را عوض کنید، همهٔ وابسته‌ها دوباره کامپایل می‌شوند. و `:app` با هر تغییری باز هم dex و بسته‌بندی را تکرار می‌کند.

سؤال تکمیلی

«*بعد از ماژول‌بندی build تمیز کندتر شد. شکست خوردیم؟*» — لزوماً نه. با build افزایشی برای تغییرهای معمول و زمان CI با cache گرم قضاوت کنید — منتظر ماندن واقعی توسعه‌دهنده‌ها همین‌جاست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. Play Feature Delivery: گزینه‌ها

نوع تحویل	زمان نصب	مناسب برای
install-time	همراه اپ	کد ماژولاری که همیشه لازم است
conditional	هنگام نصب، اگر دستگاه شرط را داشته باشد: سخت‌افزار، حداقل SDK، کشور	AR و فیچرهای وابسته به سخت‌افزار
on-demand	وقتی اپ درخواستش کند	فیچرهای بزرگ، اختیاری و کم‌استفاده

برای **asset**های بزرگ (مدل‌های ML، دادهٔ بازی) به‌جایش از **Play Asset Delivery** استفاده کنید: asset packهای install-time، fast-follow و on-demand — بدون نیاز به شکستن کد.

```
// app/build.gradle.kts — the base module
android {
    dynamicFeatures += setOf(":videocall")
}

// videocall/build.gradle.kts
plugins { id("com.android.dynamic-feature") }
dependencies { implementation(project(":app")) }
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تعریف نوع تحویل

```
<!-- videocall/src/main/AndroidManifest.xml -->
<manifest
    xmlns:dist="http://schemas.android.com/apk/distribution">
    <dist:module dist:title="@string/title_videocall">
        <dist:delivery>
            <dist:on-demand />
        </dist:delivery>
        <dist:fusing dist:include="true" />
    </dist:module>
</manifest>
```

- `dist:title` به کاربر نشان داده می‌شود، پس ماژول‌های on-demand به آن نیاز دارند
- `dist:fusing` تعیین می‌کند ماژول داخل universal APK قرار بگیرد یا نه

تلهٔ مصاحبه

«*تحویل شرطی را در* `build.gradle.kts` *تنظیم کنید.*» (جواب نسخهٔ پارسال) — شرط‌ها در **manifest** همان ماژول تعریف می‌شوند: یک بلوک `dist:conditions` داخل `dist:install-time`، با شرط‌هایی مثل `dist:device-feature`، `dist:min-sdk` یا `dist:user-countries`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. نصب در لحظهٔ نیاز

```
// import com.google…SplitInstallSessionStatus as Status
val splits = SplitInstallManagerFactory.create(context)

val listener = SplitInstallStateUpdatedListener { state ->
    when (state.status()) {
        Status.DOWNLOADING -> showProgress(state)
        Status.REQUIRES_USER_CONFIRMATION -> splits
            .startConfirmationDialogForResult(state, launcher)
        Status.INSTALLED -> openVideoCall()
        Status.FAILED -> showError(state.errorCode())
    }
}
splits.registerListener(listener) // unregister when done
val request = SplitInstallRequest.newBuilder()
    .addModule("videocall")
    .build()
splits.startInstall(request)
```

- در activityهای ماژول `SplitCompat.installActivity(this)` را صدا بزنید (و در `Application` پایه `SplitCompat.install` را) تا کد جدید بدون ری‌استارت قابل‌استفاده باشد

تلهٔ مصاحبه

«*با کتابخانهٔ Play Core دانلودش کنید.*» — کتابخانهٔ یکپارچهٔ Play Core منسوخ شده و با targetSdk 34 و بالاتر سازگار نیست. از `com.google.android.play:feature-delivery` استفاده کنید (و کتابخانه‌های جداگانهٔ Asset Delivery، In-App Updates و In-App Review).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. هزینه‌های واقعی dynamic feature

هزینه	چرا
وابستگی وارونه	ماژول پایه نمی‌تواند به کلاس‌های ماژول ارجاع دهد: نقطهٔ ورود را با اسم بارگذاری کنید (reflection یا `ServiceLoader`) و در قواعد R8 نگهش دارید
DI	Hilt نمی‌تواند dynamic feature را پردازش کند — Dagger به‌همراه `@EntryPoint` (سؤال ۶)
تست	اجرای معمولی debug همه‌چیز را نصب می‌کند؛ تحویل واقعی را با `bundletool --local-testing` یا internal app sharing تست کنید
توزیع	فقط Google Play ماژول‌ها را on-demand تحویل می‌دهد؛ فروشگاه‌های دیگر و نصب مستقیم به universal APK نیاز دارند
تجربهٔ کاربر	پیشرفت دانلود، تأیید، خطا، آفلاین، کمبود فضا — برای هر نقطهٔ ورود

اول این‌ها را امتحان کنید: R8 و resource shrinking به‌همراه Appهای split Bundle (روز ۱۸)؛ Play Asset Delivery یا سرور خودتان برای assetهای بزرگ؛ ماژول‌های کتابخانه‌ای معمولی برای نظم کد.

> **تلهٔ مصاحبه**
>
> *«ماژول Settings را از قبل دانلود کن، چون بیشتر کاربرها بازش می‌کنند.»* — اگر بیشتر کاربرها لازمش دارند، اصلاً نباید on-demand باشد. on-demand برای فیچرهایی است که اقلیتی از کاربرها استفاده می‌کنند؛ `deferredInstall()` هم فقط یک نصب پس‌زمینهٔ بدون تضمین است.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

حاکمیت بر گراف، مهاجرت از مونولیت، دانه‌بندی و اپ‌های دمو



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. حاکمیت بر گراف ماژول‌ها

قاعده	چرا
هیچ `:impl` به `:impl` دیگری وابسته نیست	فیچرها مستقل build می‌شوند
هیچ `:core:*` به `:feature:*` وابسته نیست	پایهٔ پایدار؛ بدون چرخهٔ پنهان از راه helperها
ماژول‌های `:api` سبک می‌مانند (بدون Room و Compose UI)	کامپایل مصرف‌کننده‌ها ارزان می‌ماند
سقفی برای ارتفاع گراف	مسیر بحرانی کوتاه، موازی‌سازی بیشتر

```
// app/build.gradle.kts — fails on a forbidden edge
plugins { id("com.jraska.module.graph.assertion") }

moduleGraphAssert {
    maxHeight = 5
    restricted = arrayOf(
        ":feature:.*:impl -X> :feature:.*:impl",
        ":core:.* -X> :feature:.*",
    )
}
```

دستور `./gradlew assertModuleGraph` را در CI اجرا کنید تا قاعده، pull request را fail کند — نه اینکه در review یادآوری شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. جعبه‌ابزار حاکمیت

ابزار	چه چیزی را اعمال می‌کند
modules-graph-assert	یال‌های مجاز و ممنوع، ارتفاع گراف
Dependency Analysis Gradle Plugin	وابستگی‌های بلااستفاده یا اشتباه تعریف‌شده
convention pluginها (روز ۲۱)	همهٔ ماژول‌ها یک‌جور پیکربندی می‌شوند
فایل `CODEOWNERS` برای هر ماژول	تیم مالک، ماژول خودش را review می‌کند
Konsist (روز ۳)	قواعد کد داخل ماژول‌ها

تلهٔ مصاحبه

«*قواعد ماژول را با Dependency-Check چک کنید.*» (توصیهٔ نسخهٔ پارسال) — OWASP Dependency-Check کتابخانه‌ها را برای آسیب‌پذیری‌های شناخته‌شده (CVE) اسکن می‌کند. هیچ چیزی دربارهٔ گراف ماژول‌های شما نمی‌داند.

سؤال تکمیلی

«*یعنی یک فیچر هرگز نباید به فیچر دیگر وابسته باشد؟*» — هرگز به `:impl` آن. وابستگی به `:api` آن اشکالی ندارد — قرارداد دقیقاً برای همین است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. مهاجرت از مونولیت

- **۱. اول اندازه بگیرید:** زمان build برای تغییرهای معمول؛ گراف وابستگی فعلی packageها را رسم کنید.
- **۲. ماژول را ارزان کنید:** version catalog و convention plugin، تا ساختن ماژول جدید چند دقیقه طول بکشد (روز ۲۱).
- **۳. packageهای داخل `app:` را باز کنید:** package بر اساس فیچر، حذف چرخه‌ها، و اعمال قواعد با ابزاری مثل Konsist در CI. بعد از آن، بردن packageهای تمیز به ماژول بخش آسان کار است.
- **۴. اول برگ‌ها را جدا کنید:** `:core:model`، `:core:designsystem`، `:core:network` — کدی که به هیچ چیز خاص اپ وابسته نیست.
- **۵. بعد data، بعد فیچرها، یکی‌یکی:** از جدیدترین یا کم‌گره‌ترین شروع کنید.
- **۶. ماژول `app:` به یک پوسته کوچک می‌شود؛** باقی‌مانده‌ها می‌توانند موقتاً در یک ماژول `:legacy` بمانند.

تلهٔ مصاحبه

«ماژول‌بندی را روی یک branch طولانی انجام بده و بعد merge کن.» — ماه‌ها merge conflict در حالی که کار فیچرها ادامه دارد. روی trunk انجامش دهید: هر pull request یک جداسازی، و هر کدام قابل‌انتشار.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. شکستن چرخه‌ها

```
// Before: profile opens OrderHistoryScreen and orders
// reads ProfileRepository — a cycle between two modules.
// After: profile navigates through :app (Q5), and orders
// sees only an interface — :feature:profile:api
interface ProfileReader {
    fun observeDisplayName(): Flow<String>
}

// :feature:order:impl depends on :feature:profile:api only
class OrderHeaderViewModel @Inject constructor(
    profile: ProfileReader,
) : ViewModel()
```

- **وارونگی وابستگی:** طرف پایین‌تر interface را تعریف می‌کند، طرف دیگر پیاده‌اش می‌کند و DI آن را bind می‌کند
- یا بخش مشترک را به یک ماژول جدید و پایین‌تر ببرید که هر دو بتوانند به آن وابسته شوند

تلهٔ مصاحبه

«*با بردن همهٔ* `Utils` *به* `:core:common` *شروع کن.*» — فقط کشوی خرت‌وپرت را جابه‌جا کرده‌اید، و حالا همهٔ ماژول‌ها به آن وابسته‌اند. بر اساس هدف جدا کنید، یا helper را همان‌جایی که استفاده می‌شود نگه دارید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. دانه‌بندی و اپ‌های دمو

خیلی درشت	خیلی ریز
یک ماژول برای چند فیچر — بیشتر تغییرها آن را دوباره می‌سازند	یک ماژول برای هر صفحه یا کلاس — سربار هر ماژول غالب می‌شود
چند تیم در یک ماژول به هم می‌خورند	پیمایش سخت؛ قرارداد همه‌جا

- **یک ماژول فیچر یعنی یک قابلیت با مالکیت یک تیم** — همان مرز یک context در domain
- وقتی چند تیم یک ماژول را تغییر می‌دهند جدایش کنید؛ وقتی دو ماژول همیشه با هم تغییر می‌کنند ادغامشان کنید

```
// feature/cart/demo/build.gradle.kts
plugins { id("com.android.application") }

dependencies {
    implementation(project(":feature:cart:impl"))
    implementation(project(":core:testing")) // fakes
}
```

یک **اپ دمو برای هر فیچر** فقط همان چیزی را build می‌کند که فیچر لازم دارد و مستقیم با دادهٔ fake داخل همان فیچر باز می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. چیزی که مصاحبه‌گر می‌خواهد بشنود

تلهٔ مصاحبه

«*ما ۸۰۰ ماژول داریم، پس معماری‌مان عالی است.*» — تعداد ماژول چیزی را ثابت نمی‌کند. سؤال‌های اصلی این‌هاست: طولانی‌ترین مسیر گراف چقدر است، مالک هر ماژول کیست، و یک تغییر معمولی فیچر چند ماژول را درگیر می‌کند؟

سؤال تکمیلی

«*از کجا می‌فهمید ماژول‌بندی جواب داده؟*» — زمان build افزایشی برای تغییرهای معمول، مدت CI، تعداد ماژول‌هایی که یک pull request معمولی درگیر می‌کند، و زمان review هر تیم را قبل و بعد دنبال کنید.

جواب سینیور در یک جمله

«در امتداد مالکیت و تغییر ماژول‌بندی کن: چیزهایی که با هم تغییر می‌کنند کنار هم، چیزهایی که نباید همدیگر را بشناسند با یک `api` از هم جدا — و قواعد گراف را CI چک کند، نه حافظهٔ آدم‌ها.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۵ — خلاصهٔ یک‌صفحه‌ای

- ماژول‌ها قواعد معماری را به قواعد کامپایلر تبدیل می‌کنند: `internal`، `implementation` و یال‌های مجاز.
- برد اصلی در build افزایشی، موازی‌سازی و cache است — build تمیز شاید سریع‌تر نشود. اندازه بگیرید.
- انواع ماژول: app، فیچر، data، core، تست و dynamic feature هستند. ماژول `:core` را بر اساس هدف تقسیم کنید — نه کشوی خرت‌وپرت.
- وابستگی‌ها یک‌طرفه‌اند؛ هیچ فیچری به `:impl` فیچر دیگر وابسته نیست.
- ماژول `:api` یعنی routeها و interfaceها، کوچک و پایدار. و **ربطی** به configurationای به نام `api()` در Gradle ندارد.
- ناوبری: فیچرها route و graph builder منتشر می‌کنند؛ `:app` آن‌ها را با lambda به هم وصل می‌کند.
- در DI، bindingها در `:impl` زندگی می‌کنند و Hilt آن‌ها را در `:app` جمع می‌کند. در dynamic feature باید از Dagger به‌همراه `@EntryPoint` استفاده کرد.
- کلاس R غیرگذرا: هر ماژول مالک resourceهای خودش است؛ `resourcePrefix` جلوی تداخل بی‌صدا را می‌گیرد.
- Play Feature Delivery: سه حالت install-time، conditional و on-demand — شرط‌ها در manifest، با کتابخانهٔ Play Feature Delivery، نه Play Core.
- گراف را در CI کنترل کنید؛ مهاجرت را تدریجی و روی trunk انجام دهید، از برگ‌ها شروع کنید.

روز ۲۶ در راه است: Kotlin پیشرفته. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700